

Vision Term Project —Rashad's Tasks

Bonus deployment + final polish

May 17, 2026

Contents

Overview	1
1. Required: Bonus deployment tasks (the +5 % grade)	2
Task 1.1 —Build & verify the Docker image (~10 min)	2
Task 1.2 —Deploy Gradio app to Hugging Face Spaces (~20 min)	2
Task 1.3 —(Optional, +polish) Push Docker image to Docker Hub	3
2. Required: Final submission polish	4
Task 2.1 —Convert paper to PDF (~5 min)	4
Task 2.2 —Add the deployment screenshots to the paper (~5 min)	4
Task 2.3 —Add the public Gradio URL to abstract + Section 7 (~1 min)	4
Task 2.4 —(Optional) Add a second author + acknowledgements line	4
3. Required: Presentation deck (~1-2 hours)	4
4. Quick reference	5
Where the best model lives	5
How to run a quick local prediction	5
How to launch the Gradio app locally (for testing before HF)	6
How to launch the FastAPI service locally	6
5. TL;DR —the critical path	6

Overview

Hey Rashad! All the heavy CV work is done:

- **Phase 1 (classical HOG + SVM):** re-run on our exact splits + 55.97 % test accuracy
- **Phase 2 (CNN study):** three models trained, evaluated, robustness-tested
- **Best model:** BaselineCNN + augmentation + **96.89 % top-1 / 0.958 macro-F1** on test
- **Paper, notebook, ONNX/CoreML exports, Gradio/FastAPI code, Dockerfile:** all in this folder

You now own the **Bonus Option A —Deployment** finish line and the final submission polish. Below is exactly what's left.

The full reproduction guide is in `HANDOFF.md`. This file is the prioritised punch list.

1. Required: Bonus deployment tasks (the +5 % grade)

Everything is coded —these are the “make it real” steps that take the code from local-only to actually deployed.

Task 1.1 —Build & verify the Docker image (~10 min)

```
cd VisionProject_handoff
docker build -t mtsd-api -f src/deploy/Dockerfile .
docker run --rm -p 8000:8000 mtsd-api
```

In another terminal:

```
# Liveness check
curl -s http://localhost:8000/healthz
# Expected: {"status":"ok"}
```

```
# Service info
curl -s http://localhost:8000/ | python3 -m json.tool
```

```
# Real prediction (any sign image works)
curl -X POST -F "file=@some_sign.jpg" http://localhost:8000/predict
```

The image is ~600 MB compressed (CPU-only PyTorch). It bakes the trained model (results/best_model.pt) inside, so it's fully self-contained.

Verification screenshot to grab: terminal showing the prediction JSON response. Goes into the paper's Section 7 and the slides.

Task 1.2 —Deploy Gradio app to Hugging Face Spaces (~20 min)

This is what the grader actually clicks and sees. Highest-impact step.

1. Sign up at <https://huggingface.co> (free).
2. Click “**New Space**” □ name it e.g. mtsd-traffic-sign-classifier, SDK = **Gradio**, hardware = CPU basic (free).
3. Clone the new Space repo locally:

```
git clone https://huggingface.co/spaces/<your-username>/mtsd-traffic-sign-classifier
cd mtsd-traffic-sign-classifier
```

4. Copy these files **from the handoff folder** into the Space repo:

From	To (Space root)
src/deploy/app.py	app.py
src/deploy/inference.py	src/deploy/inference.py
src/__init__.py	src/__init__.py
src/config.py	src/config.py
src/data.py	src/data.py
src/models/__init__.py	src/models/__init__.py
src/models/baseline_cnn.py	src/models/baseline_cnn.py
src/deploy/__init__.py	src/deploy/__init__.py
results/best_model.pt	results/best_model.pt
data/label_map.json	data/label_map.json
data/splits.json	data/splits.json (<i>only if data.py references it; if app crashes for label names you can stub it</i>)

5. Create a tiny requirements.txt in the Space root:

```
torch
torchvision
numpy
Pillow
h5py
gradio==4.44.1
huggingface_hub<1.0
```

6. Commit + push:

```
git add . && git commit -m "Initial MTSD classifier deploy"
git lfs install      # (probably needed for the .pt file)
git lfs track "*.pt"
git add .gitattributes
git commit -m "lfs track .pt"
git push
```

7. The Space builds + serves automatically. Public URL will be <https://huggingface.co/spaces/<your-username>/mtsd-traffic-sign-classifier>.

Verification screenshot to grab: the live Gradio app correctly classifying a sign with the top-5 confidence bars.

Task 1.3 —(Optional, +polish) Push Docker image to Docker Hub

```
docker login
docker tag mtsd-api <your-dockerhub-user>/mtsd-api:latest
docker push <your-dockerhub-user>/mtsd-api:latest
```

Means anyone can do `docker pull <your-dockerhub-user>/mtsd-api` to run the service. Optional but a nice “ship it” gesture.

2. Required: Final submission polish

These touch the paper and presentation deliverables.

Task 2.1 — Convert paper to PDF (~5 min)

```
cd VisionProject_handoff/paper
pandoc paper.md -o paper.pdf --pdf-engine=xelatex \
  --metadata title="MTSD Traffic Sign Classification" \
  --metadata author="<both names>" \
  -V geometry:margin=1in -V fontsize=11pt
```

If pandoc complains about missing fonts on your machine, swap `--pdf-engine=xelatex` for `--pdf-engine=pdflatex`. If neither work, run `pandoc paper.md -o paper.html` then “Print → Save as PDF” from a browser.

Task 2.2 — Add the deployment screenshots to the paper (~5 min)

Open `paper/paper.md`, find **Section 7 Deployment**, and replace the text bullets with two mark-down images:

![Live Gradio demo on Hugging Face Spaces](figs/gradio_screenshot.png)

![FastAPI Swagger UI at /docs](figs/fastapi_screenshot.png)

Drop the screenshots into `paper/figs/` first. Re-export to PDF.

Task 2.3 — Add the public Gradio URL to abstract + Section 7 (~1 min)

Just paste the HF Space URL where the paper currently says `<huggingface-space-url>`.

Task 2.4 — (Optional) Add a second author + acknowledgements line

Currently the paper header says `Author: <name>`. Replace with both names, course code, and date.

3. Required: Presentation deck (~1-2 hours)

The course has a 30-min slot. Suggested slide outline:

#	Slide	What to show
1	Title + team	Project name, both authors, date
2	Problem statement	MTSD overview, 400+ classes, long-tail

#	Slide	What to show
3	Dataset curation	Why drop other-sign, <30 threshold; final 326 classes
4	Phase 1 —classical	HOG □ SGD-SVM pipeline diagram, 55.97 % result
5	Phase 2 —custom CNN architecture	The 5-row table from the paper
6	Training curves	Use re— sults/baseline/training_curves.png
7	Augmentation experiment	Before/after augmentation accuracy bars
8	Transfer learning	EfficientNet-B0 comparison
9	Headline results table	The 4-row table
10	Robustness	Use re— sults/aug/robustness.png (line plot)
11	Confusion matrix	Use re— sults/aug/test_confusion_matrix.png
12	Deployment	Screenshots + the Gradio URL (live demo if possible!)
13	Live demo	Switch to a browser □ drop in 2-3 sample signs
14	Limitations + future work	From paper’s Section 8
15	Thank you / questions	

All the figures already exist in `results/` —just screenshot them into PowerPoint or Keynote.

The executed notebook (`notebooks/visionPhase2.html`) is also a great “slide source”—every figure is rendered there.

4. Quick reference

Where the best model lives

```
results/best_model.pt      ← 27 MB, BaselineCNN + augmentation
results/aug/best.pt       ← same file (canonical run name)
results/aug/export/model.onnx ← cross-platform export
results/aug/export/model.mlpackage ← Apple Core ML
```

How to run a quick local prediction

```
.venv/bin/python -c "
from src.deploy.inference import SignClassifier
from PIL import Image
clf = SignClassifier('results/best_model.pt')
```

```
preds = clf.predict(Image.open('SOME_SIGN.jpg'))
for p in preds:
    print(f'{p.probability:.3f}  {p.label}')
```

How to launch the Gradio app locally (for testing before HF)

```
.venv/bin/python -m src.deploy.app
# opens http://localhost:7860
```

How to launch the FastAPI service locally

```
.venv/bin/uvicorn src.deploy.api:app --host 127.0.0.1 --port 8000
# Swagger docs at http://127.0.0.1:8000/docs
```

5. TL;DR —the critical path

If you only do **two things**, do these:

1. **Build the Docker image** and verify it serves predictions (proves the Docker bonus).
2. **Deploy the Gradio app to Hugging Face Spaces** and grab the public URL (the grader will actually use it).

Total time: ~30 minutes. After that, everything else is paper/slides polish.

Good luck —ping me if anything in `HANDOFF.md` is unclear.